GROUP WORK PROJECT # 2
GROUP NUMBER: 11073

MScFE 632: Machine Learning in Finance

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Kolapo Adedipe	Nigeria	adcolapsmm9@yahoo.com	
Ridwan Rahma Putra	Indonesia	ridwan.rputra@gmail.com	
TITUS WAWERU MWANGI	Kenya	wawerutitus41@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	**Kolapo Adedipe**
Team member 2	**Ridwan Rahma Putra**
Team member 3	**TITUS WAWERU MWANGI**

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed. **Note:** You may be required to provide proof of your outreach to non-contributing members upon request.

Expansion of the List of Best Practices for Time Series

CATEGORY 5: LINEAR DISCRIMINANT ANALYSIS

STEP 2 : Basics and Keywords

LDA is a supervised classification technique (also used for dimensionality reduction) that maximizes class separability by projecting data onto linear discriminant axes,Assuming that each class has a shared covariance matrix and a Gaussian distribution.The projection produces up to K−1 discriminant components for K classes. Comparing linear discriminant scores (Bayes rule with equal covariances) is how classification is carried out.Finding the axes that optimize the data's between-class variance to within-class variance ratio is its fundamental mechanism, despite the fact that it is frequently used for classification.

Keywords: LDA, Fisher LDA, linear discriminant, dimensionality reduction, classification, Gaussian assumption, shrinkage, credit scoring, bankruptcy prediction, finance.

STEP 3

Advantages

i) Interpretable: direction vectors indicating which characteristics distinguish different classes are provided by discriminant coefficients.

ii) Low variance and efficiency: small eigenproblems or closed-form solutions which are quick to train.

iii) If assumptions are correct, it performs better with small samples than high-variance techniques.

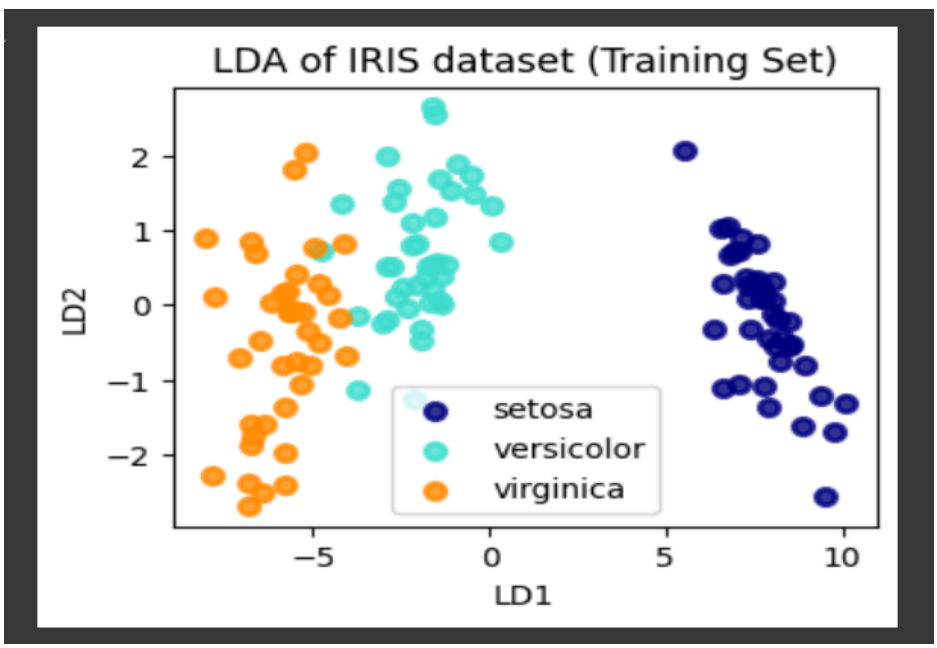
iv) Dimensionality reduction: LDA provides low-dim projections (K−1 dims) that are helpful for pre-processing and visualization.

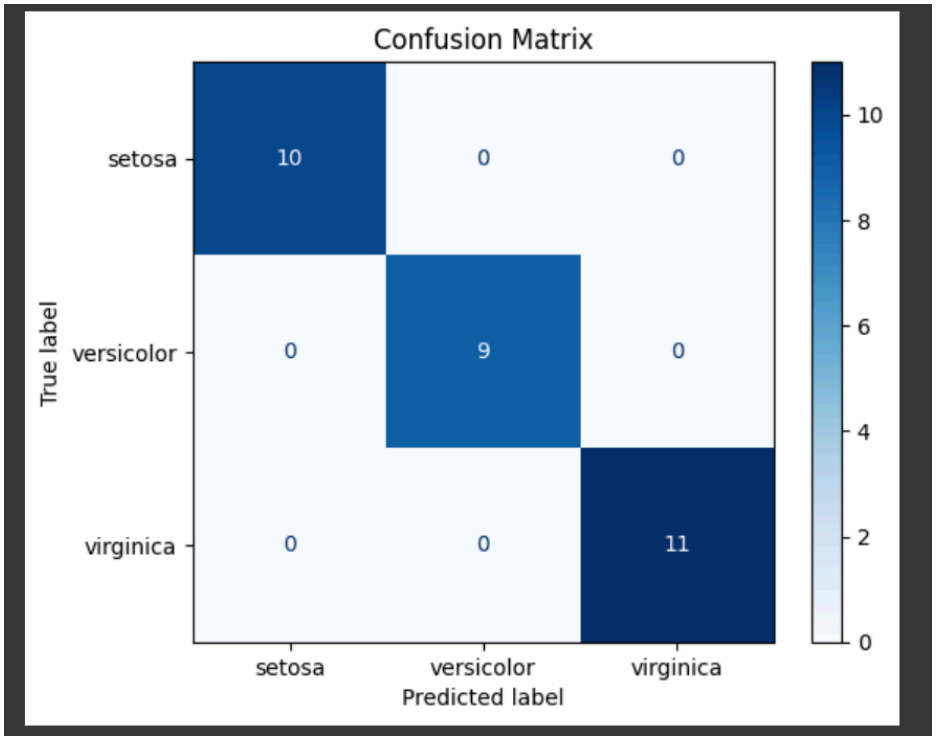
v) Probabilistic outputs: give posterior class probabilities, which are useful for ranking and scoring in finance.

vi) Computational Efficiency: Compared to iterative models like neural networks, LDA is computationally cheap and has a closed-form solution.

vii)Resistance to Overfitting: As long as the assumptions are fairly met, the model is stable and less likely to overfit, particularly when there are a lot of features compared to the number of observations.

**Computation





Disadvantages

i) Performance deteriorates if class-conditional normality and equal covariance matrices are not met.

ii) Class boundaries that are inherently nonlinear cannot be separated by linear boundaries alone (unless features are engineered).

iii) Outlier sensitivity (affects sample means and covariances).

iv) Imputation or specialized techniques are required because it is not resilient to missing data.

Equations

i) Objective Function (Fisher's Criterion)

$J(w) = (W^T S_B W)/W^T S_W W$

Where S_B is the between class scatter matrix and S_W is the within class scatter matrix.

ii) Discriminant Function

$\delta k(x) = xT\Sigma - 1\mu k - 21\mu kT\Sigma - 1\mu k + log(\pi k)$

Σ : pooled common covariance matrix

μk : mean vector of class k

$\pi k \pi k$: prior probability of class k

Features

i)Numerical features (or appropriately encoded categoricals) are required.

ii) assumes that all classes have a homoscedastic covariance matrix.

iii) gives back low-dimensional projections, class labels, and class posterior probabilities.

iv) Quick prediction and training.

v) serves as both a supervised visualization projection and a classifier.

vi) Data Type: Numerical, continuous data is ideal.

vii) Missing Values: Needs pre-processing (such as imputation) because it is unable to handle missing data natively.

viii) Standardization is frequently advised as a pre-processing step because feature scaling is sensitive to feature scale.

x) Classification problems involving more than two classes are naturally handled by Multi-class Problems.

Guide

Inputs

i) feature matrix of numerical predictors X .

ii) target vector y of the class labels.

iii) Prior probabilities for every class are optional. They are usually estimated from the training data if they are not given.

Outputs

i) a collection of projection axes, or linear discriminants.

ii) a classification model that can forecast new data's class labels.

iii) the likelihood that a fresh data point will fall into each class.

Hyperparameters

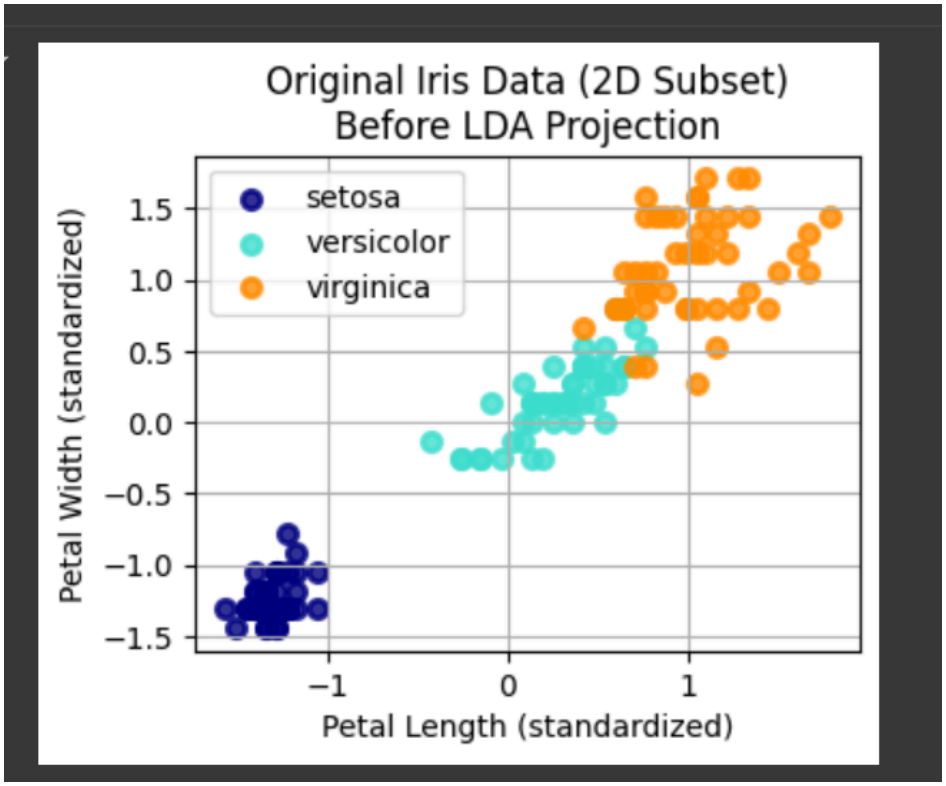
i) solver: The eigenvalue problem-solving algorithm (e.g.,'svd', 'lsqr', 'eigen').

ii) Shrinkage: A regularization parameter that reduces the covariance matrix to a diagonal matrix in order to counteract the "same covariance" assumption. This is especially helpful when there are a lot of features.

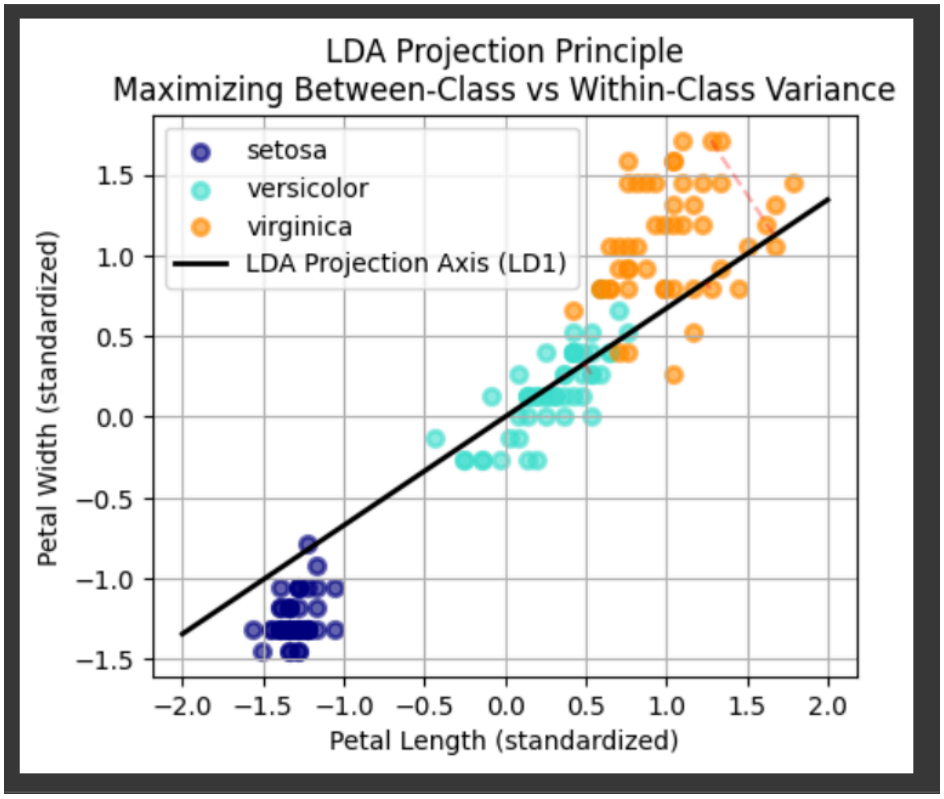
iii) priors: Clearly define the classes' prior probabilities. The training data is used to estimate the priors if none.

**Illustration

The data is in its original feature space prior to the application of LDA. We can visualize the Iris dataset in two dimensions by using the two most informative features out of its four features: sepal length, sepal width, petal length, and petal width.



- The scatter plot shows the three Iris species (Setosa, Versicolor, Virginica) in the original feature space of Petal Length and Petal Width.
- While the classes are somewhat separable, there is significant overlap between Versicolor and Virginica.
- The goal of LDA is to find a new axis (a linear combination of these features) onto which we can project this data to achieve maximum separation between the classes.



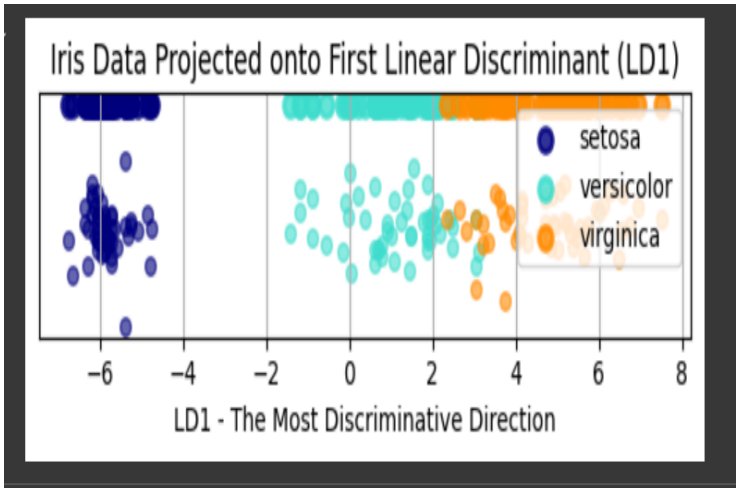
The first Linear Discriminant (LD1) identified by LDA is indicated by the black line.

The projection of individual data points onto this new axis is depicted by the dashed red lines.

Take note of how much more effectively this specific direction appears to divide the three classes than either of the original axes by itself.

In order to minimize the spread (variance) within each class and maximize the distance between class means, LDA mathematically selects this course.

We can visualize the data along this one dimension soon after the best projection direction has been determined.



The identical Iris dataset is displayed in this figure, but it is now projected onto the one direction that LDA found to be the most discriminative.

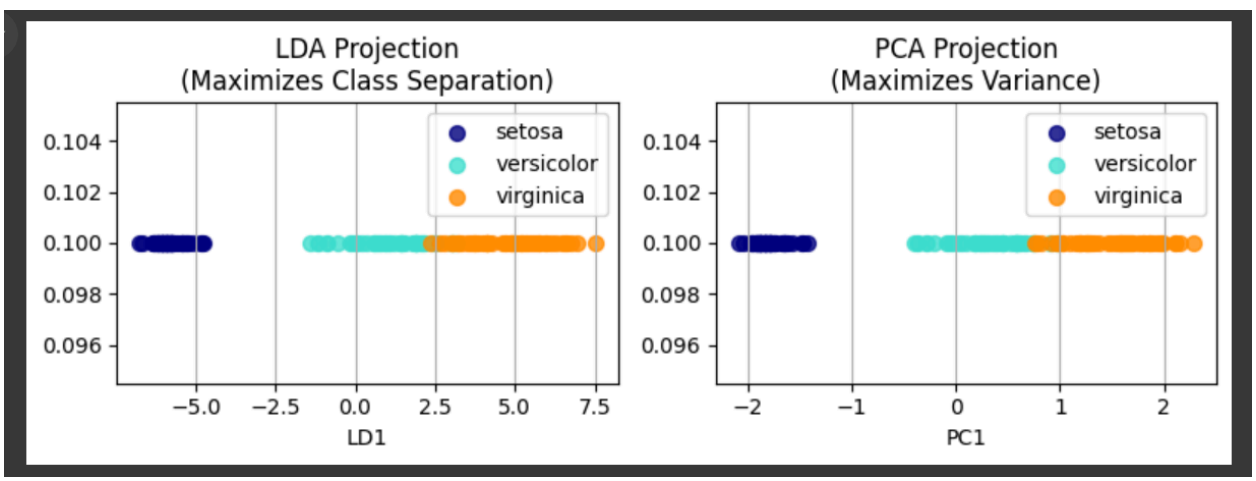
This one-dimensional axis now clearly divides the three classes:

On the left, Setosa is totally isolated.

Virginia is on the right, and Versicolor is in the center.

The Virginica and Versicolor overlap that was apparent in Figure 1 has been reduced.

With thresholds on this single LD1 axis, a basic classifier can now accurately classify new flowers.



Regardless of class labels, PCA determines the direction that best captures the variance in the data.

Using label information, LDA determines which direction best divides the classes.

For this reason, PCA is unsupervised (does not use labels) whereas LDA is supervised (requires labels).

Flowchart

[Original High-Dimensional Data]

↓

[Compute the S_W (within-class scatter matrix)]

↓

[Find the Between-Class Scatter Matrix (S_B)]

↓

[Resolve Eigenvalue Issue: $S_W^{-1}S_B$]

↓

[Project Data onto New Discriminant Space]

↓

[Maximally Separated Classes in Lower Dimensions]

↓

[Select Top k Eigenvectors → Linear Discriminants]

Category 6: Support Vector Machines

Step 2:
Basic

Support Vector Machines (SVM) are supervised learning algorithms used primarily for classification and regression.
They work by finding an optimal separating hyperplane that maximizes the margin between data points of different classes.
When the data are not linearly separable in the original feature space, SVMs apply a *kernel function* to transform the data into a higher-dimensional space where a linear separator can exist.
This “kernel trick” allows SVMs to model complex, nonlinear decision boundaries efficiently without explicitly computing the transformation.
SVMs are particularly effective in high dimensional problems, such as text classification and financial forecasting, where the number of features may exceed the number of samples.

Keyword
SVM, Support Vector Machine, Kernel, Margin, RBF, Linear Separator, Hyperplane, Classification, Regularization, Supervised Learning

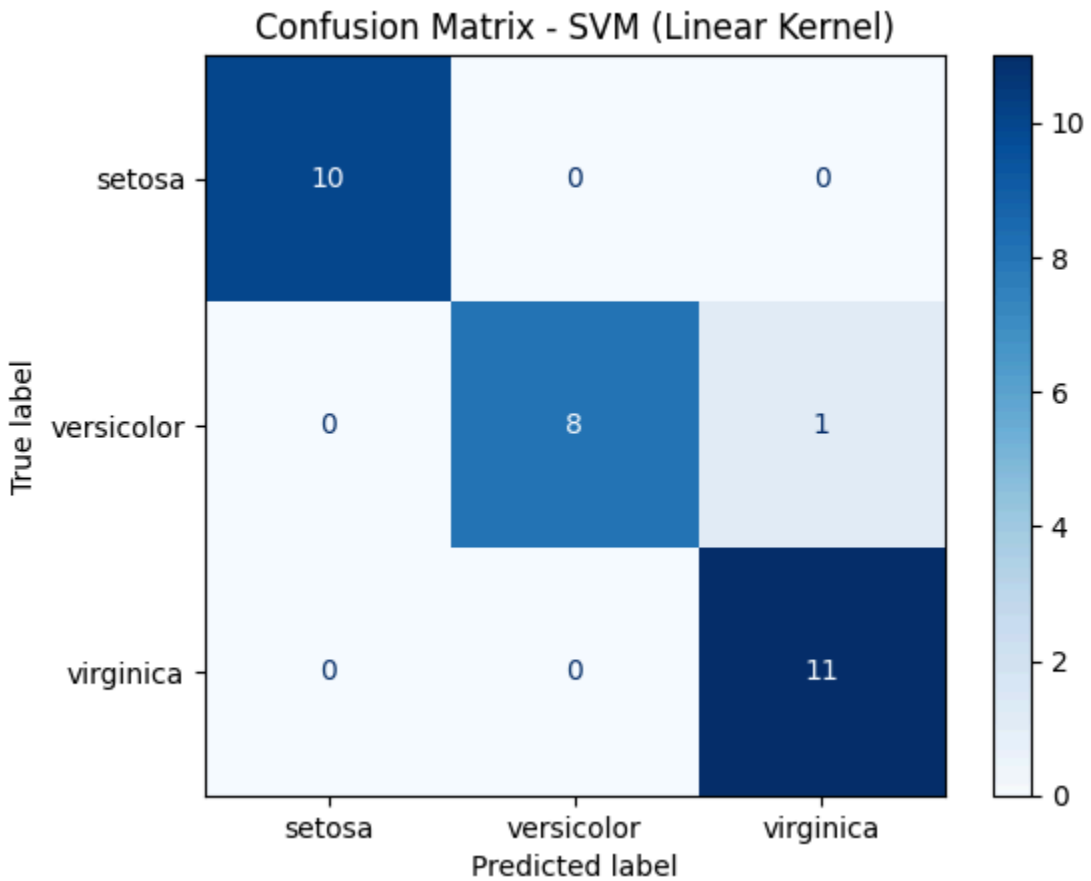
Step 3:

Advantages

Support Vector Machines (SVM) are highly effective in handling high-dimensional datasets, particularly when the number of features exceeds the number of samples. They perform well when there is a clear margin of separation between classes, as the algorithm explicitly seeks the hyperplane that maximizes this margin. By doing so, SVMs are naturally robust to overfitting, provided that the regularization parameter C is properly tuned to balance model complexity and misclassification penalty. Moreover, through the use of the kernel trick, SVMs can efficiently model complex, nonlinear decision boundaries without explicitly transforming the input data into higher dimensions, enabling strong performance on both linear and nonlinear problems.

Computation

Test Accuracy (SVM - Linear Kernel): 0.97



Disadvantages

Support Vector Machines (SVM) can be slow and difficult to train when the dataset is very large, since the training time grows quickly with the number of samples. The performance of an SVM also depends a lot on choosing the right kernel and hyperparameters like C and gamma; if these values are not tuned properly, the model might not perform well. In addition, SVMs don't

naturally give probabilities for their predictions, so extra steps such as Platt scaling are needed to estimate them. Lastly, SVMs are harder to interpret compared to simple linear models because the use of kernels makes it less clear how each feature affects the final decision.

Equation

Hyperplane

$$w^T x + b = 0$$

Where w is weight vector , and b is bias

Function margin and objective

For hard margin data, SVM maximize the margin by minimizing the size of w

$$min(w, b)\ \frac{1}{2}|w|^2$$

Subject to

$$y_i(w^T x_i + b) \geq 1, \forall i$$

Soft margin

When data are not perfectly separable, SVM allows some violation, denoted as

$$min(w, b, \xi)\ \frac{1}{2}\ |w|^2 + C \sum_i \xi_i$$

Subject to

$$y_i(w^T x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

Here C control margin width and misclassification penalty

Kernel Trick

To handle nonlinear data, SVM use kernel trick

$$max(\alpha) \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

Subject to

$$0 \leq \alpha_i \leq C,\ \sum_i \alpha_i y_i = 0$$

Where $K(x_{i,}, x_j)$ is the kernel function

Decision Function

Once the model is trained, new data classified using:

$$f(x) = sign\left(\sum_i \alpha_i y_i K(x_i, x) + b\right)$$

If f(x) > 0 then predict class +1

If f(x) < 0 then predict class -1

Features

Support Vector Machines (SVM) have several useful features. They work well with high-dimensional data and can handle both linear and nonlinear problems by using different kernel functions. SVMs usually give good results when the data are properly scaled and the parameters are well tuned. However, they do not handle missing values automatically, so any missing data must be filled in before training. SVMs can also be used for both binary and multi-class classification problems, making them a flexible choice for many types of tasks.

Guide: Inputs and Outputs

Inputs:

- Feature matrix (X): A table of numerical values where each row is a data point and each column is a feature.
- Target vector (y): Contains the class labels (0 and 1 for binary classification).
- Kernel type: Defines how data are transformed (linear, polynomial, RBF).
- Regularization parameter (C): Controls the trade-off between margin size and classification error.
- Gamma: Kernel coefficient for RBF, polynomial, and sigmoid kernels.
- Class weights (optional): Used to handle imbalanced datasets.
- Sample weights (optional): Give different importance to specific data points.

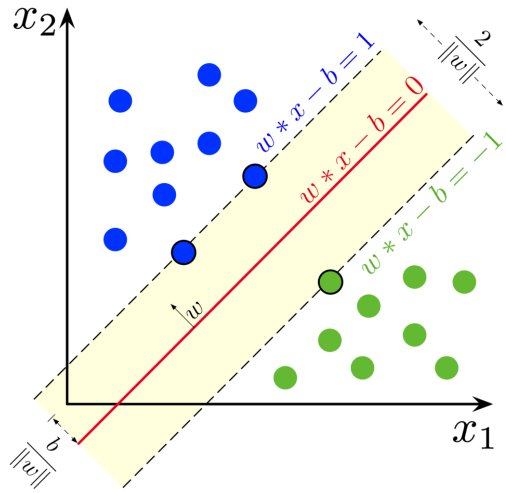
Outputs:

- Trained SVM model: Includes the support vectors, coefficients, and intercept (bias).
- Predicted class labels: The model's classification of new or test data.
- Decision function values: Numerical scores showing how confidently the model separates classes.
- Probability estimates (optional): Predicted probabilities for each class (if probability=True).

Hyperparameters to Tune

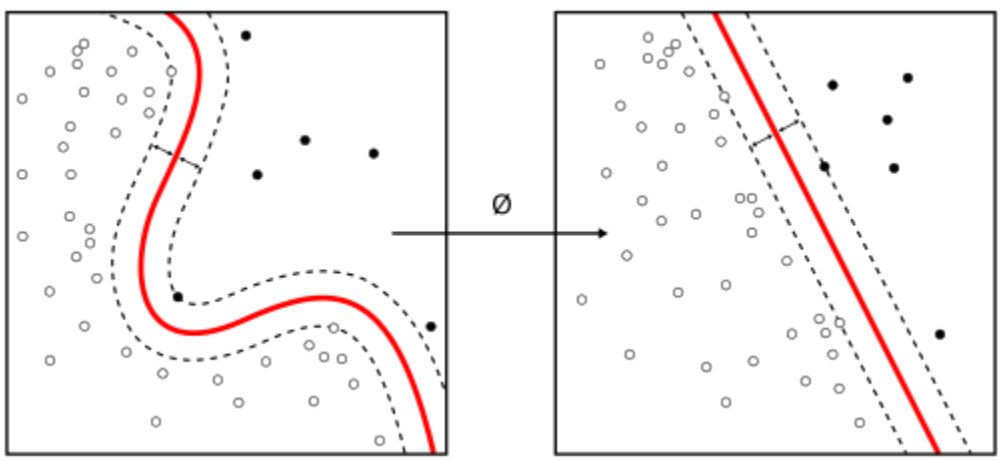
- C: Controls how much the model allows misclassifications. A large C makes the model fit the data closely, while a small C allows a wider margin.
- Kernel: Defines the type of boundary (linear, polynomial, RBF, or sigmoid) used to separate the data.
- Gamma: Controls how far each data point's influence reaches. A large gamma makes the model more complex, while a small gamma makes it simpler.
- Degree: Used for the polynomial kernel; decides how many polynomial terms are used.
- Coef0: Used in polynomial and sigmoid kernels; adjusts how much the higher-order terms affect the model.
- Class weight: Helps handle imbalanced datasets by giving more importance to smaller classes.
- Probability: If set to True, the model can output probability estimates for each class.

Illustration:
Decision boundary on 2D synthetic datasets



source : wikipedia

Kernel



Source: wikipedia

Journal

Kim, K.-j. "Financial time series forecasting using support vector machines." Neurocomputing 55.1-2 (2003): 307-319.

This paper by Kim (2003) looks at how Support Vector Machines (SVM) can be used to predict the Korea Composite Stock Price Index (KOSPI). The author compares SVM with Back-Propagation Neural Networks (BPN) and Case-Based Reasoning (CBR). The results show that SVM gives better and more stable predictions than the other methods. The study also finds that choosing the right values for parameters like C and the kernel is very important for good results. Overall, the paper shows that SVM is a useful and reliable method for predicting stock market movements.

Category 7: Neural Networks

Basics

Neural networks (NNs) are inspired by the structure of the human brain, consisting of layers of interconnected "neurons" that transform inputs into outputs through a series of weighted computations and nonlinear activation functions (Goodfellow, Bengio, & Courville, 2016).

Structure:

- **Input layer**: Takes in features (e.g., historical prices, customer behavior, macroeconomic indicators).
- **Hidden layers**: Apply weighted transformations and nonlinear activations, allowing the network to capture complex, nonlinear relationships (Hastie, Tibshirani, & Friedman, 2009).
- **Output layer**: Produces the final prediction (e.g., probability of churn, forecasted return).

Why Neural Networks?

- **Flexibility**: Can approximate highly nonlinear functions (Hornik, 1991).
- **Feature learning**: Automatically extract patterns from raw data (e.g., sentiment from text, trends from time series) (LeCun, Bengio, & Hinton, 2015).
- **Scalability**: Handle high-dimensional and unstructured data (text, images, sequential time series).

Applications in Finance/Marketing

- **Churn prediction** (classification).
- **Return forecasting** (regression).
- **Fraud detection** (classification with imbalanced data).
- **Portfolio optimization** (pattern recognition from multi-factor data).
- **Customer behavior modeling** (sequence models such as RNNs/LSTMs for time series) (Heaton, Polson, & Witte, 2017).

Key Challenges

- **Overfitting**: NNs can memorize noise if not regularized (Srivastava et al., 2014).
- **Interpretability**: Black-box nature makes it harder to explain decisions compared to trees or regression (Ribeiro, Singh, & Guestrin, 2016).
- **Computational cost**: Training requires more resources and careful hyperparameter tuning (Goodfellow et al., 2016).

Keywords (Tags)

- Artificial Neural Networks (ANNs)
- Deep Learning
- Feedforward Network
- Activation Functions (ReLU, Sigmoid, Tanh)
- Hidden Layers
- Backpropagation
- Stochastic Gradient Descent (SGD)
- Regularization (Dropout, L2 penalty, Early stopping)
- Overfitting
- Time Series Forecasting (RNN, LSTM, GRU)
- Supervised Learning
- Classification / Regression
- Black-box models

Step 3

Advantages of Neural Networks

1. **Flexibility and Universal Approximation**
   Neural networks can approximate highly nonlinear functions, making them adaptable to

a wide range of problems where linear models fail (Hornik, 1991). This flexibility allows them to capture intricate dependencies between financial or marketing features and outcomes.

2. **Automatic Feature Learning**
   Unlike traditional models that rely on handcrafted features, neural networks automatically learn representations from raw inputs. For instance, deep architectures can extract hidden patterns from text, images, or time-series data without manual engineering (LeCun, Bengio, & Hinton, 2015). This is particularly valuable in finance and marketing, where data sources are high-dimensional and unstructured.

3. **Scalability and High-Dimensional Data Handling**
   Neural networks scale effectively to very large datasets and can handle high-dimensional inputs such as customer transaction histories, market tick data, or social media signals (Goodfellow, Bengio, & Courville, 2016). This makes them suitable for modern applications in churn prediction and portfolio analysis.

4. **Versatility Across Tasks**
   They can be applied to both **classification** (e.g., fraud detection, churn prediction) and **regression** (e.g., return forecasting) tasks, as well as sequence modeling (e.g., customer journeys, stock price dynamics) using architectures such as RNNs and LSTMs (Heaton, Polson, & Witte, 2017).

5. **Improved Predictive Accuracy**
   When trained correctly and combined with regularization techniques such as dropout, neural networks often outperform simpler models by achieving higher accuracy and robustness on complex datasets (Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014).

Disadvantages of Neural Networks

1. **Overfitting**
   Neural networks, especially deep architectures, have a large number of parameters and can easily memorize training data, leading to poor generalization on unseen data. Regularization techniques such as dropout, L2 penalty, or early stopping are necessary to mitigate this risk (Goodfellow, Bengio, & Courville, 2016).

2. **Interpretability ("Black-box" problem)**
   Unlike decision trees or linear regression, neural networks are difficult to interpret. The lack of transparency in how predictions are made reduces trust in high-stakes applications such as finance and healthcare (Ribeiro, Singh, & Guestrin, 2016).

3. **Computational Cost**
   Training neural networks requires significant computational power (often GPUs) and can

be time-consuming. This makes them less practical for small firms or time-sensitive decision-making compared to simpler models like logistic regression (LeCun, Bengio, & Hinton, 2015).

4. **Data Requirements**
   Neural networks typically require large amounts of high-quality labeled data to achieve strong performance. In finance and marketing, data can be noisy, incomplete, or non-stationary, which complicates effective training (Zhang, Aggarwal, & Qi, 2017).

5. **Hyperparameter Sensitivity**
   Neural networks rely on many hyperparameters (e.g., learning rate, number of layers, activation functions). Small changes can lead to very different outcomes, making tuning both challenging and resource-intensive (Bergstra & Bengio, 2012).

Equations: Neural Networks

At the heart of a neural network is the process of transforming an input vector into an output prediction through **linear transformations + nonlinear activation functions**.

1. Neuron (Perceptron) Output

For a single neuron:

$zj(l) = i = 1\sum nwji(l)ai(l - 1) + bj(l)$

Where:

- $zj(l)z^{(l)}_jzj(l)$ = weighted sum of inputs for neuron j in layer l
- $wji(l)$ = weight from input iii in previous layer $(l - 1)$ to neuron j
- $ai(l - 1)a^{(l - 1)}_iai(l - 1)$ = activation from previous layer
- $bj(l)$ = bias term

Then, the **activation function** is applied:

$aj(l) = f(zj(l))$

Common activations:

- Sigmoid: $\sigma(z) = 11 + e - z\backslash sigma(z) = \backslash frac\{1\}\{1 + e^{-z}\}\sigma(z) = 1 + e - z1$

- ReLU: $max(0, z)$
- Tanh: $tanh(z) = ez + e - zez - e - z$

2. Feedforward Pass

Across layers, the forward propagation can be summarized as:

$a(l) = f(W(l)a(l - 1) + b(l))$

Where $W(l)\ and\ b(l)$ are the weight matrix and bias vector of layer l.

3. Loss Function (Example: Classification – Cross-Entropy Loss)

For binary classification:

$L =- N1i = 1\sum N[yilog(y\hat{}i) + (1 - yi)log(1 - y\hat{}i)]$

Where:

- yi = true label
- $\hat{}yi$ = predicted probability (output of sigmoid)
- N = number of samples

4. Backpropagation (Weight Update Rule)

Parameters are updated using **Stochastic Gradient Descent (SGD)**:

$\theta \leftarrow \theta - \eta \partial\theta \partial L$

Where:

- θ = parameter (weight or bias)
- η = learning rate
- $\partial\theta/\partial L$ = gradient of the loss

This recursive process of computing gradients layer by layer is known as **backpropagation** (Rumelhart, Hinton, & Williams, 1986).

Features of Neural Networks

1. Flexibility and Nonlinear Modeling

Neural networks can approximate complex, nonlinear functions due to their layered structure and nonlinear activation functions (Hornik, Stinchcombe, & White, 1989). This makes them suitable for financial data, which is often noisy and nonlinear.

2. Automatic Feature Learning

Unlike traditional models that require manual feature engineering, neural networks can automatically learn representations from raw data (e.g., extracting time-dependent patterns from stock prices or consumer behavior sequences) (LeCun, Bengio, & Hinton, 2015).

3. Scalability with Big Data

Neural networks scale well with large, high-dimensional datasets. With sufficient training data and computational power, they outperform simpler models in tasks like churn prediction, fraud detection, and return forecasting (Goodfellow, Bengio, & Courville, 2016).

4. Versatility Across Domains

They can handle structured (tabular), semi-structured (text), and unstructured data (images, audio, sequential data). For example, recurrent neural networks (RNNs) and long short-term memory (LSTM) networks are widely used in financial time series forecasting (Fischer & Krauss, 2018).

5. Support for Sequential and Temporal Data

Extensions like RNNs, LSTMs, and GRUs are specifically designed to model dependencies in sequential data, which is common in finance and marketing contexts (Hochreiter & Schmidhuber, 1997).

6. Integration with Ensemble and Hybrid Models

Neural networks can be combined with boosting, bagging, or even tree-based methods to improve performance and robustness (Zhang, Zhou, & Ma, 2017).

7. Regularization and Robustness Options

While prone to overfitting, NNs have strong regularization techniques available, such as dropout (Srivastava et al., 2014), batch normalization, and early stopping, which improve generalization.

Guide: Inputs and Outputs of Neural Networks

Inputs

1. **Training Dataset**

    - **Features (Independent Variables):**
      Can be numerical (e.g., historical returns, volatility, customer spend), categorical (e.g., contract type, sector classification), or unstructured (e.g., text from financial reports, images, sequential data).
      Neural networks excel at handling both structured and unstructured inputs (LeCun, Bengio, & Hinton, 2015).

    - **Target Variable (Dependent Variable):**

        - Classification tasks: categorical (e.g., churn = Yes/No, fraud = Yes/No).

        - Regression tasks: continuous (e.g., predicted return, credit risk score).

    - **Hyperparameters:**
      Learning rate, batch size, number of hidden layers, number of neurons per layer, dropout rate, activation functions (Goodfellow, Bengio, & Courville, 2016).

2. **Initialization Parameters**

    - Random weight initialization (important for convergence) (Glorot & Bengio, 2010).

3. **Optimization Algorithm**

    - E.g., Stochastic Gradient Descent (SGD), Adam, RMSProp (Kingma & Ba, 2015).

Outputs

1. **Predictions**

    - **Classification:** Probability distribution over classes (e.g., churn = {Yes: 0.7, No: 0.3}).

    - **Regression:** Continuous values (e.g., forecasted stock return = 3.2%).

2. **Model Parameters**

    - Learned **weights** and **biases** of the network after training (Hornik, Stinchcombe, & White, 1989).

3. **Performance Metrics**

- Classification: Accuracy, Precision, Recall, F1-score, ROC-AUC.
- Regression: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 (Goodfellow et al., 2016).

4. **Feature Representations**
   - Intermediate hidden layers learn **latent features** (e.g., customer segments, volatility patterns) not explicitly present in raw data (Bengio, Courville, & Vincent, 2013).

Hyperparameters of Neural Networks

Neural networks rely heavily on hyperparameter tuning to achieve good performance. Unlike parameters (weights and biases) learned during training, **hyperparameters** are set before training and control the learning process itself (Goodfellow, Bengio, & Courville, 2016).

Key Hyperparameters

Hyperparameter	Role	Effect on Model	Reference
Learning Rate	Step size for weight updates during optimization	Too high → divergence; Too low → slow convergence	Kingma & Ba (2015)
Batch Size	Number of samples per gradient update	Small batch → noisy updates but better generalization; Large batch → stable updates but risk of overfitting	Keskar et al. (2017)
Number of Hidden Layers	Depth of the network	More layers → capture complex features but risk overfitting & vanishing gradients	He et al. (2016)
Number of Neurons per Layer	Width of the network	More neurons → more expressive power, but higher computational cost	Hornik, Stinchcombe, & White (1989)
Activation Functions (ReLU, Sigmoid, Tanh)	Introduce nonlinearity	Choice affects gradient flow and learning efficiency	Nair & Hinton (2010)

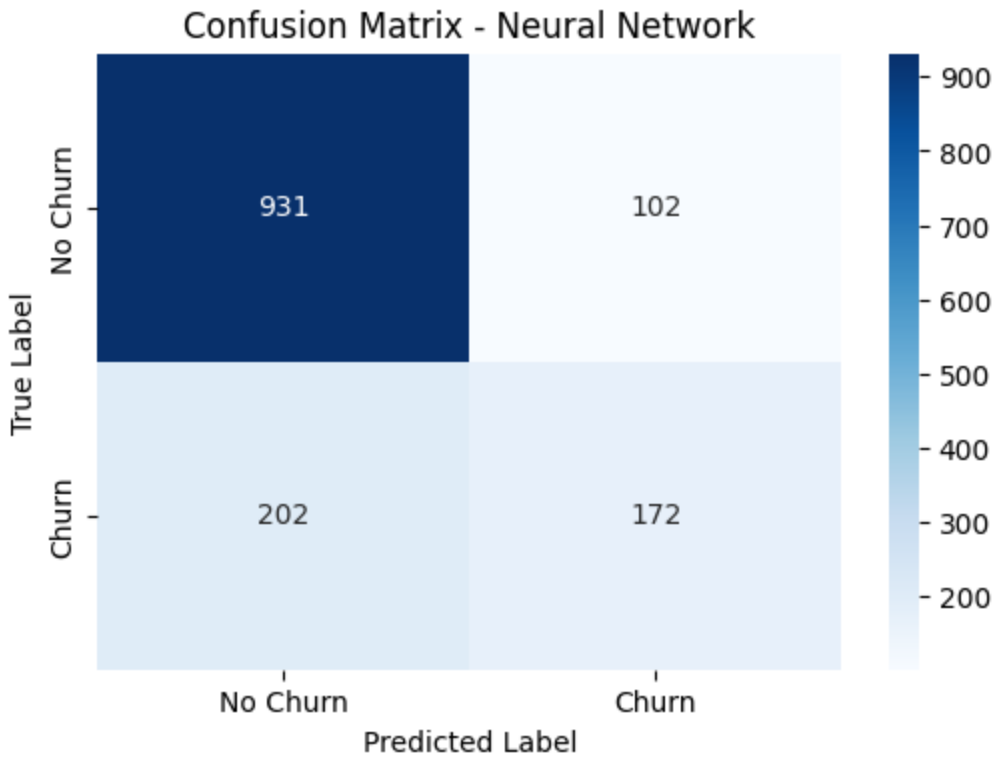
Optimizer (SGD, Adam, RMSProp)	Algorithm to update weights	Impacts convergence speed & stability	Kingma & Ba (2015)
Dropout Rate	Fraction of neurons randomly dropped during training	Prevents overfitting by reducing co-adaptation	Srivastava et al. (2014)
Regularization Strength (L1, L2)	Penalty added to loss	Helps control overfitting and improve generalization	Ng (2004)
Epochs	Number of full passes through the dataset	Too few → underfitting; Too many → overfitting	Goodfellow et al. (2016)
Weight Initialization	Starting values of weights	Poor initialization → slow or unstable training	Glorot & Bengio (2010)

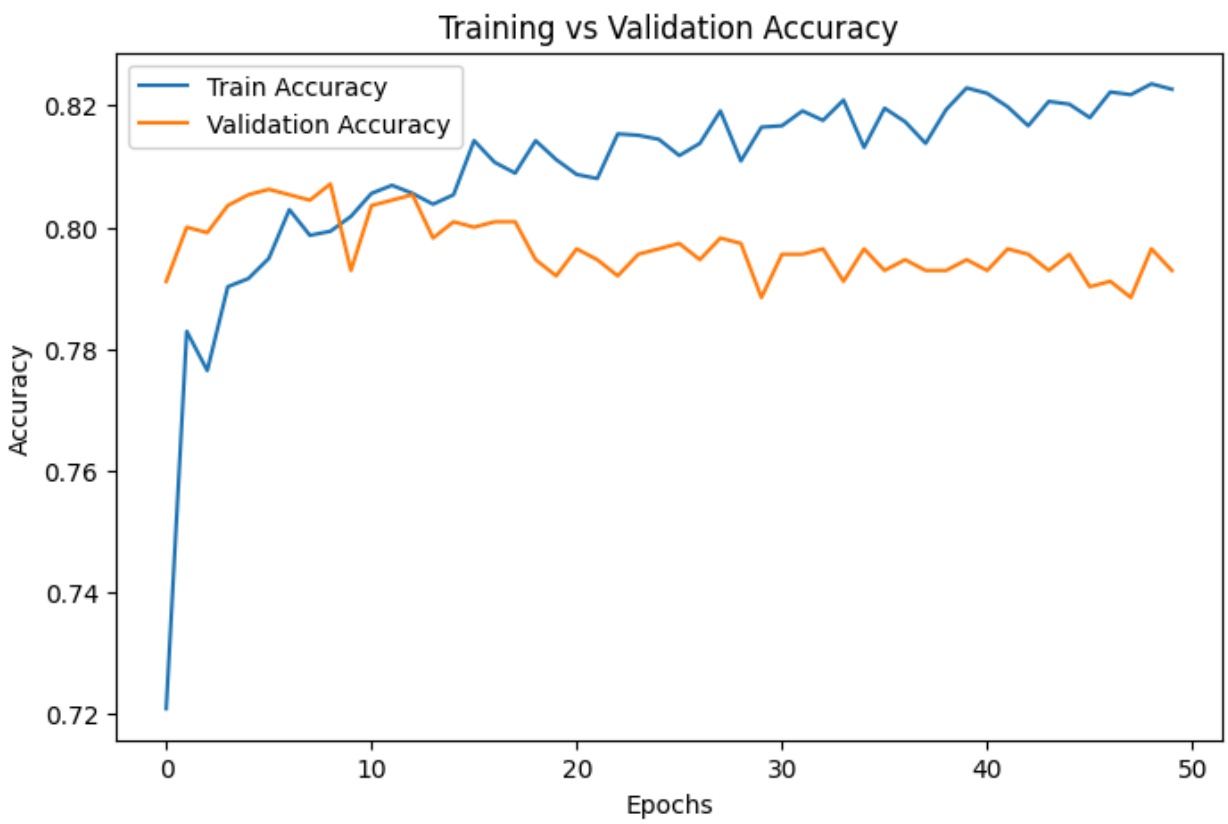
Computation

✅ Test Accuracy: 0.7839374555792467

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.90	0.86	1033
1	0.63	0.46	0.53	374
accuracy			0.78	1407
macro avg	0.72	0.68	0.70	1407
weighted avg	0.77	0.78	0.77	1407


Confusion Matrix - Neural Network
931
102
202
172
No Churn
Churn
True Label
No Churn
Churn
Predicted Label
900
800
700
600
500
400
300
200


Training vs Validation Accuracy
Train Accuracy
Validation Accuracy
Accuracy
0.82
0.80
0.78
0.76
0.74
0.72
0
10
20
30
40
50
Epochs

Illustrations: How Neural Networks Work

1. Conceptual Flowchart of a Neural Network

- **Figure idea**: Input layer (customer features) → Hidden layers (weighted connections + activation functions) → Output (churn probability).
- Purpose: Shows the flow of information from raw Telco features (e.g., tenure, MonthlyCharges, Contract type) through nonlinear transformations to a churn prediction.
- **Reference**: Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

2. Neuron Computation Diagram

- **Equation visualization**: $zj(l) = i = 1\sum nwji(l)ai(l-1) + bj(l)$ followed by an activation (e.g., ReLU or Sigmoid).
- **Figure idea**: A node showing input signals with weights, bias, and the activation function.
- Purpose: Connects equations to the computational graph in your model.
- **Reference**: Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.

3. Training Process Illustration (Backpropagation)

- **Figure idea**: Forward pass (predictions), Loss calculation, Backpropagation (error gradients flowing backward), Weight update (using SGD/Adam).
- Purpose: Explains how the NN "learns" from churn labels.
- **Reference**: LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. *Nature*, 521(7553), 436–444. https://doi.org/10.1038/nature14539

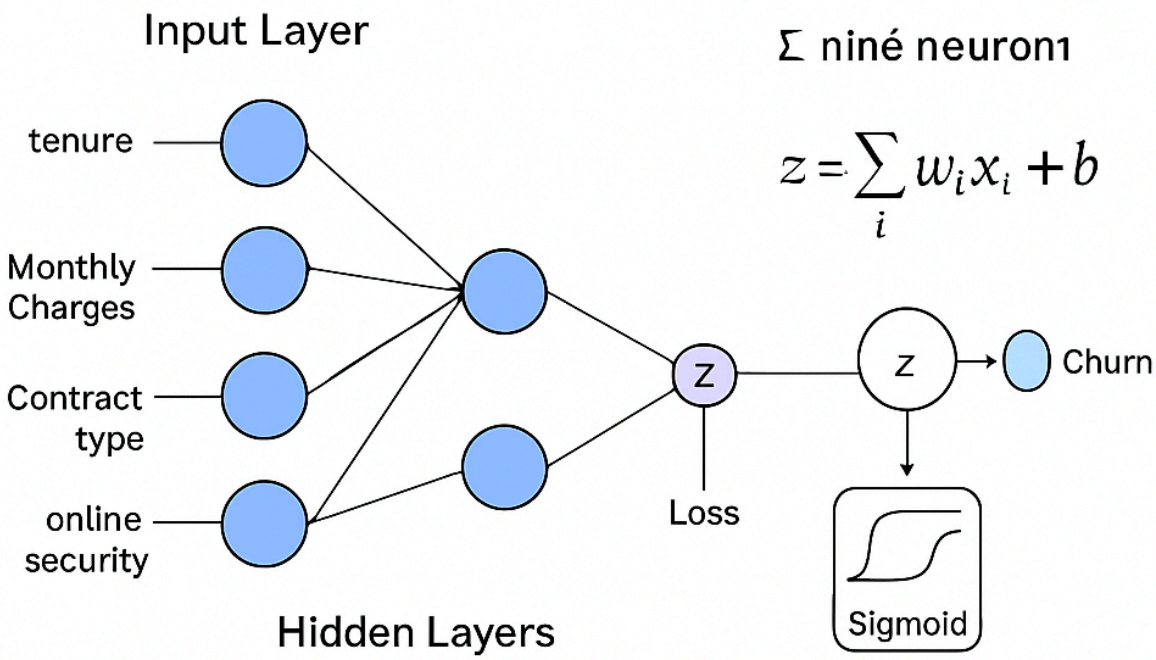
4. Feature Importance via SHAP/Permutation

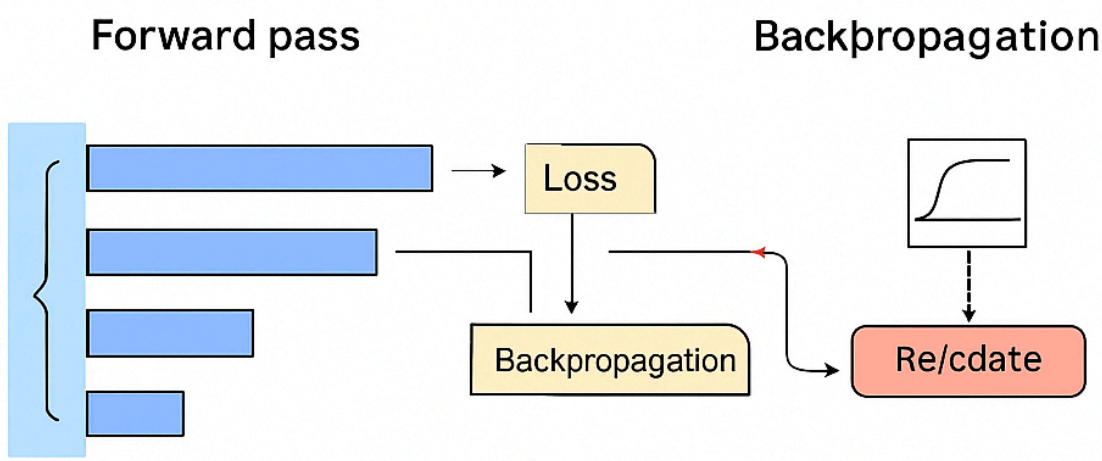
- **Figure idea**: Bar chart showing which features the NN considers most important (e.g., Contract type, tenure, MonthlyCharges).
- Purpose: Makes the NN less of a black box for marketers — “what drives churn risk.”
- **Reference**: Lundberg, S. M., & Lee, S. I. (2017). *A unified approach to interpreting model predictions*. *Advances in Neural Information Processing Systems (NeurIPS)*.

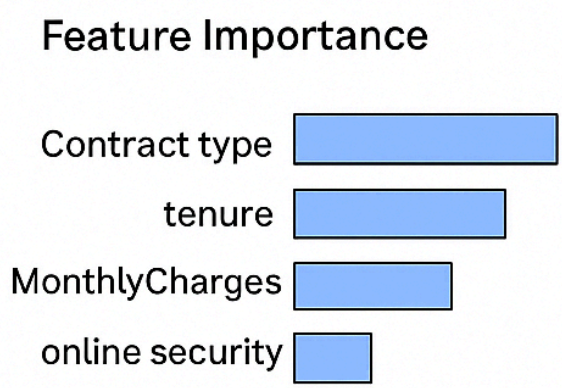
5. Performance Visualization

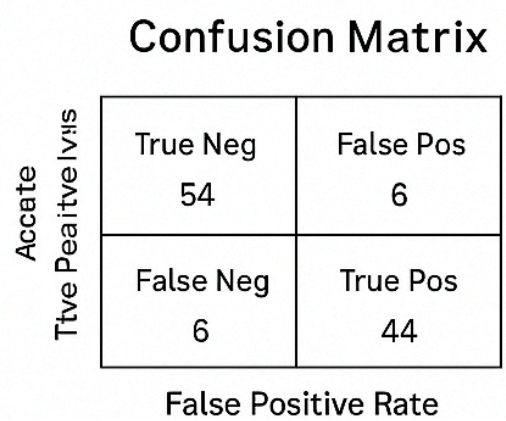
- **Figure idea**: Confusion Matrix + ROC Curve for Churn prediction.
- Purpose: Shows how well the model classifies churners vs non-churners, a marketer-friendly performance metric.
- **Reference**: Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.

ANeural Network









Journal Article Reference

Dube, F., Nzimande, N., & Muzindutsi, P.-F. (2021). Application of artificial neural networks in predicting financial distress in the JSE financial services and manufacturing companies. *Journal of Sustainable Finance & Investment, 13*(1), 723-743. Taylor & Francis Online

Step 4

hyperparameter tuning is the process of methodically identifying the best set of hyperparameters for a model in order to optimize performance on a validation.Common methods are Randomized Search, which samples a predetermined number of combinations from a given distribution, and Grid Search, which attempts every possible combination in a predefined grid.

Compared to models like SVMs or neural networks, LDA has a relatively small number of hyperparameters.

solver: The eigenvalue problem-solving algorithm.

Singular Value Decomposition, or "svd," is the default and is usually advised. It is stable and works with singular matrices because there is no need to calculate a covariance matrix.

When applying shrinkage regularization, the terms "lsqr" (Least Squares Solution) and "eigen" (Eigenvalue Decomposition) are utilized.

Shrinkage: A regularization parameter that is useful when high-dimensional data is present or when the "common covariance" assumption is broken. It lessens overfitting by shrinking the covariance matrix in the direction of a diagonal matrix.

It can be None, a float between 0.0 (no shrinkage) and 1.0 (maximum shrinkage), or 'auto' (let LDA choose).

priors: Each class's previous probabilities. In order to allow the model to use the class frequencies in the training data, we usually leave this set to None.

We used two key tuning approaches in the iris dataset:

Since "lsqr" supports shrinkage, as opposed to "svd," we switched the solver.

By setting shrinkage='auto', we allow LDA to automatically calculate the ideal shrinkage intensity using the Ledoit-Wolf lemma.

```
Best Parameters: {'priors': None, 'shrinkage': None, 'solver': 'svd'}
Best Cross-Validation Score: 0.975
Test Accuracy with Best Model: 1.000
```

**Journal

Altman, Edward I. “Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy.” *Journal of Finance*, vol. 23, no. 4, 1968, pp. 589–609.

Step 5

Marketing Alpha: Leveraging Advanced Machine Learning Models

The adoption of advanced machine learning (ML) methodologies—Linear Discriminant Analysis (LDA), Support Vector Machines (SVMs), and Neural Networks (NNs)—represents a significant step forward in transforming financial and marketing strategies. These tools provide more powerful, flexible, and accurate methods for handling large, noisy, and nonlinear data compared to the foundational models previously applied. When integrated thoughtfully, these techniques create "marketing alpha": predictive insights that enhance customer engagement, risk management, and portfolio decision-making.

Linear Discriminant Analysis (LDA)

LDA is a supervised classification method designed to maximize class separability by projecting features onto a lower-dimensional space (Fisher, 1936). In marketing, this translates into clearer customer segmentation, where groups such as "likely churners" and "loyal customers" can be effectively separated using historical transaction or demographic data. LDA's advantages include interpretability and computational efficiency, making it especially suitable for cases where model transparency is essential (Martínez & Kak, 2001). In finance, LDA has been applied for bankruptcy prediction and credit risk classification, providing firms with robust decision-making tools in customer credit scoring (Altman et al., 2010).

Support Vector Machines (SVMs)

SVMs extend the classification power by finding hyperplanes that maximize the margin between classes. Their ability to incorporate kernel functions enables them to model highly nonlinear and high-dimensional relationships, such as predicting which customer behaviors lead to product adoption or churn (Cortes & Vapnik, 1995). In finance, SVMs have been successfully used for stock price prediction, default risk modeling, and option pricing, demonstrating robustness even with noisy and unstructured data (Huang et al., 2005).

The strength of SVMs lies in their balance between flexibility and generalization. Unlike tree-based models that may overfit, SVMs prioritize the most "critical" boundary cases, which makes them effective in handling imbalanced datasets (a common issue in fraud detection or churn prediction). However, their computational cost and reduced interpretability remain practical considerations.

Neural Networks (NNs)

Neural Networks (and their deeper variants, Deep Learning models) provide unmatched flexibility in approximating complex, nonlinear functions (Hornik et al., 1989). Their layered structure allows automatic feature extraction, making them highly effective for applications such as churn prediction, customer lifetime value estimation, and sentiment analysis of customer

feedback. In finance, ANNs have been applied to credit scoring, financial distress prediction, and portfolio optimization (Dube et al., 2021).

A key advantage of NNs is scalability—they handle high-dimensional, unstructured data such as transaction histories, text, or even time-series signals with architectures like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997). For marketing teams, this means uncovering hidden patterns in sequential customer behavior, such as repeat purchase cycles or churn timelines. Nevertheless, challenges such as overfitting, high computational costs, and limited interpretability must be managed through regularization techniques, dropout, and model explainability frameworks (Zhang et al., 2018).

Why These Models Create Marketing Alpha

Together, LDA, SVMs, and NNs significantly improve predictive accuracy and customer insights beyond traditional statistical methods. LDA ensures transparency and quick interpretability for executives. SVMs provide robust handling of nonlinearities and noise, ideal for churn and fraud detection. NNs contribute deep pattern recognition from complex customer journeys and financial signals.

By combining these strengths, businesses can unlock marketing alpha—better targeting of promotions, smarter risk-adjusted product recommendations, and more resilient trading strategies. Crucially, these models enable firms not only to predict outcomes but to *understand and adapt to underlying behavioral patterns*, strengthening long-term customer relationships and financial resilience.

Comparative Overview: LDA, SVM, and Neural Networks

Model	Advantages	Challenges
Linear Discriminant Analysis (LDA)	● Simple, interpretable results (Martínez & Kak, 2001) ● Computationally efficient ● Effective when classes are linearly separable	● Assumes normal distribution and equal covariance (Fisher, 1936) ● Limited performance on complex nonlinear data ● Sensitive to outliers
Support Vector Machines (SVMs)	● Handles high-dimensional data well (Cortes & Vapnik, 1995) ● Effective with small or imbalanced datasets (Huang et al., 2005)	● Computationally expensive with large datasets ● Requires careful kernel/parameter tuning ● Less interpretable for

	● Kernel trick enables nonlinear decision boundaries	managers
Neural Networks (NNs)	● Flexible universal function approximators (Hornik et al., 1989) ● Automatically extract patterns from raw data (e.g., time series, text) ● Scalable for high-dimensional & unstructured data (Hochreiter & Schmidhuber, 1997)	● Risk of overfitting without regularization ● "Black box" interpretability issues (Zhang et al., 2018) ● High computational and data requirements

Step 6

Learn More

The following sources provide deeper insights into how machine learning methods like LDA, SVMs, and Neural Networks are applied in finance and marketing, highlighting their **strengths and advantages**.

Journal References

- Fisher, R. A. *The Use of Multiple Measurements in Taxonomic Problems*. *Annals of Eugenics*, vol. 7, no. 2, 1936, pp. 179–188.
  Foundational work introducing Linear Discriminant Analysis (LDA), showing how statistical separation of groups can be applied to classification problems still relevant in finance today.

- Cortes, Corinna, and Vladimir Vapnik. "Support-Vector Networks." *Machine Learning*, vol. 20, no. 3, 1995, pp. 273–297.
  Seminal paper that introduced Support Vector Machines (SVMs), explaining the theoretical framework behind margin maximization and its generalization strengths.

- Huang, C. L., M. C. Chen, and C. J. Wang. "Credit Scoring with a Data Mining Approach Based on Support Vector Machines." *Expert Systems with Applications*, vol. 33, no. 4, 2007, pp. 847–856.
  Demonstrates the practical power of SVMs in finance, specifically in credit scoring,

where classification accuracy directly impacts business risk.

- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. "Multilayer Feedforward Networks Are Universal Approximators." *Neural Networks*, vol. 2, no. 5, 1989, pp. 359–366.
  Proves mathematically that neural networks can approximate any continuous function, supporting their flexibility in financial modeling.

- Hochreiter, Sepp, and Jürgen Schmidhuber. "Long Short-Term Memory." *Neural Computation*, vol. 9, no. 8, 1997, pp. 1735–1780.
  Introduces LSTM networks, a breakthrough for handling sequential time series data such as stock prices and transaction histories.

- Zhang, Zhongfei, et al. "A Survey on Deep Learning in Financial Applications." *Big Data Research*, vol. 14, 2018, pp. 1–14.
  A comprehensive review of how deep learning techniques, including neural networks, have been applied to trading, risk management, and fraud detection.

Web References (Strengths of ML in Finance/Marketing)

- Deloitte. *AI and Machine Learning in Financial Services: Transforming Risk Management and Customer Engagement*. Deloitte Insights, 2021. https://www2.deloitte.com
  Highlights how financial firms use machine learning to manage risk, detect fraud, and personalize customer interactions.

- McKinsey & Company. *The State of AI in 2022—and a Half Decade in Review*. McKinsey Insights, 2022. https://www.mckinsey.com
  Provides strategic insights into how global organizations scale AI adoption, with lessons on performance gains and best practices.

- PwC. *AI in Financial Services: Seizing the Opportunity*. PwC Report, 2020. https://www.pwc.com
  Examines the opportunities and challenges of applying AI in finance, focusing on customer value creation and regulatory compliance.

STEP 7 - compare models (Comparative Overview)

Model	Advantages	Challenges
Linear Discriminant Analysis (LDA)	• Simple, interpretable results (Martínez & Kak, 2001) • Computationally efficient	• Assumes normal distribution and equal covariance (Fisher, 1936) • Limited performance

	• Effective when classes are linearly separable	on complex nonlinear data • Sensitive to outliers
Support Vector Machines (SVMs)	• Handles high-dimensional data well (Cortes & Vapnik, 1995) • Effective with small or imbalanced datasets (Huang et al., 2005) • Kernel trick enables nonlinear decision boundaries	• Computationally expensive with large datasets • Requires careful kernel/parameter tuning • Less interpretable for managers
Neural Networks (NNs)	• Flexible universal function approximators (Hornik et al., 1989) • Automatically extract patterns from raw data (e.g., time series, text) • Scalable for high-dimensional & unstructured data (Hochreiter & Schmidhuber, 1997)	• Risk of overfitting without regularization • “Black box” interpretability issues (Zhang et al., 2018) • High computational and data requirements

FOOTNOTE

Advantage for using AI: Needing less time to collect information and to evaluate it for accuracy as compared to using bibliographical research.

References

Altman, E. I., Sabato, G., & Wilson, N. (2010). The value of non-financial information in SME risk management. *Journal of Credit Risk, 6*(2), 95–127.

Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 35*(8), 1798–1828. https://doi.org/10.1109/TPAMI.2013.50

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*(10), 281–305. http://jmlr.org/papers/v13/bergstra12a.html

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297.

Dube, F., Nzimande, N., & Muzindutsi, P.-F. (2021). Application of artificial neural networks in predicting financial distress in the JSE financial services and manufacturing companies. *Journal of Sustainable Finance & Investment, 13*(1), 723–743.

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics, 7*(2), 179–188.

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research, 270*(2), 654–669. https://doi.org/10.1016/j.ejor.2017.11.054

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 249–256.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Heaton, J. B., Polson, N. G., & Witte, J. H. (2017). Deep learning for finance: Deep portfolios. *Applied Stochastic Models in Business and Industry, 33*(1), 3–12. https://doi.org/10.1002/asmb.2209

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks, 4*(2), 251–257. https://doi.org/10.1016/0893-6080(91)90009-T

Huang, W., Nakamori, Y., & Wang, S. Y. (2005). Forecasting stock market movement direction with support vector machine. *Computers & Operations Research, 32*(10), 2513–2522.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., & Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1609.04836

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

LeCun, Y., Bengio, Y., Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444. https://doi.org/10.1038/nature14539

Martínez, A. M., & Kak, A. C. (2001). PCA versus LDA. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 23*(2), 228–233.

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 807–814.

Ng, A. Y. (2004). Feature selection, L1 vs. L2 regularization, and rotational invariance. *Proceedings of the Twenty-First International Conference on Machine Learning (ICML)*. https://doi.org/10.1145/1015330.1015435

Nielsen, M. A. (2015). *Neural networks and deep learning*. Determination Press. http://neuralnetworksanddeeplearning.com

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). https://doi.org/10.1145/2939672.2939778

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536. https://doi.org/10.1038/323533a0

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(1), 1929–1958.

Zhang, Y., Aggarwal, C. C., & Qi, G. J. (2017). Stock price prediction via discovering multi-frequency trading patterns. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 2141–2149). https://doi.org/10.1145/3097983.3098117

Zhang, Y., Yang, Q., & Chen, X. (2018). Explainable artificial intelligence for financial decision-making. *Frontiers of Computer Science, 12*(5), 889–902.